

📁 BharatYatra Backend API Documentation

****Version:**** 1.0.0

****Author:**** Vishal

****Date:**** December 2025

****Project:**** BharatYatra - Explore India Platform

📑 Table of Contents

1. [Architecture Overview](#architecture-overview)
2. [Technology Stack](#technology-stack)
3. [Project Structure](#project-structure)
4. [Database Models](#database-models)
5. [API Endpoints](#api-endpoints)
6. [Authentication System](#authentication-system)
7. [Server Configuration](#server-configuration)
8. [Development Setup](#development-setup)
9. [Security Considerations](#security-considerations)
10. [Deployment Guide](#deployment-guide)

🏗️ Architecture Overview

****BharatYatra**** is a Node.js/Express backend API designed for exploring Indian states, cities, places, food, culture, and travel information. The system follows a ****RESTful API**** architecture with ****MongoDB**** as the primary database.

Key Features:

- **Multi-tier Data Structure**: States → Cities → Places/Foods/Transport
- **Admin Panel Support**: Complete CRUD operations with authentication
- **Bulk Upload System**: Excel file processing for mass data import
- **Search & Filter**: Advanced querying capabilities
- **Cultural Information**: Rich cultural data management
- **Translation Services**: Multi-language support

🛠️ Technology Stack

Component	Technology	Version	Purpose
	-----	-----	-----
Runtime	Node.js	Latest	JavaScript runtime
Framework	Express.js	^5.2.1	Web application framework
Database	MongoDB Atlas	Cloud	NoSQL database
ODM	Mongoose	^9.0.2	Object Document Mapping
Authentication	JWT	^9.0.3	JSON Web Tokens
File Upload	Multer	^2.0.2	Multipart form handling
Excel Processing	XLSX	^0.18.5	Excel file parsing
HTTP Client	Axios	^1.13.2	HTTP requests
Environment	Dotenv	^17.2.3	Environment variables
CORS	CORS	^2.8.5	Cross-origin requests

📁 Project Structure

...

server/

└─ 📁 config/

- | └─ db.js # MongoDB connection configuration
- | └─  controllers/ # Business logic layer
- | └─ stateController.js # Public state operations
- | └─ cityController.js # Public city operations
- | └─ placeController.js # Public place operations
- | └─ foodController.js # Public food operations
- | └─ transportController.js # Public transport operations
- | └─ cultureController.js # Culture data operations
- | └─ translatorController.js # Translation services
- | └─ recommendationController.js # Travel recommendations
- | └─ itineraryController.js # Itinerary generation
- | └─ adminController.js # Admin authentication
- | └─ adminStateController.js # Admin state CRUD
- | └─ adminCityController.js # Admin city CRUD
- | └─ adminPlaceController.js # Admin place CRUD
- | └─ adminFoodController.js # Admin food CRUD
- | └─ adminTransportController.js # Admin transport CRUD
- | └─ adminBulkUpload*.js # Bulk upload controllers
- | └─  middlewares/
- | └─ auth.middleware.js # JWT authentication middleware
- | └─  models/ # MongoDB schemas
- | └─ State.js # State data model
- | └─ City.js # City data model
- | └─ Place.js # Tourist place model
- | └─ Food.js # Food item model
- | └─ Transport.js # Transport option model
- | └─ Culture.js # Cultural information model
- | └─ Admin.js # Admin user model
- | └─ Recommendation.js # Travel recommendation model
- | └─ Itinerary.js # Itinerary model

- | └─ HiddenGem.js # Hidden gems model
- | └─ SafetyInfo.js # Safety information model
- | └─ CulturalQuiz.js # Cultural quiz model
- | └─  routes/ # API endpoint definitions
- | └─ stateRoutes.js # Public state routes
- | └─ cityRoutes.js # Public city routes
- | └─ placeRoutes.js # Public place routes
- | └─ foodRoutes.js # Public food routes
- | └─ transportRoutes.js # Public transport routes
- | └─ cultureRoutes.js # Culture routes
- | └─ translatorRoutes.js # Translation routes
- | └─ recommendationRoutes.js # Recommendation routes
- | └─ itineraryRoutes.js # Itinerary routes
- | └─ adminRoutes.js # Admin authentication routes
- | └─ adminStateRoutes.js # Admin state routes
- | └─ adminCityRoutes.js # Admin city routes
- | └─ adminPlaceRoutes.js # Admin place routes
- | └─ adminFoodRoutes.js # Admin food routes
- | └─ adminTransportRoutes.js # Admin transport routes
- | └─  scripts/
- | └─ createAdmin.js # Admin account creation script
- | └─  utils/
- | └─ apiResponse.js # Standardized API responses
- | └─  .env # Environment variables
- | └─  server.js # Main application entry point
- └─  package.json # Project dependencies
- ...

📖 Database Models

1. ****State Model**** (`State.js`)

```
````javascript
{
 name: {
 type: String,
 required: true,
 unique: true,
 trim: true
 },
 slug: {
 type: String,
 required: true,
 unique: true,
 lowercase: true
 },
 description: {
 type: String,
 required: true
 },
 culturalSummary: {
 type: String
 },
 image: {
 type: String, // URL
 required: true
 },
 isActive: {
 type: Boolean,
 default: true
 }
}
```

```
},
timestamps: true // createdAt, updatedAt
}
...

```

### ### 2. **\*\*City Model\*\*** (`City.js`)

```
````javascript
{
  name: {
    type: String,
    required: true,
    trim: true
  },
  slug: {
    type: String,
    required: true,
    lowercase: true
  },
  stateId: {
    type: ObjectId,
    ref: "State",
    required: true
  },
  description: {
    type: String,
    required: true
  },
  history: {
    type: String
  },
  image: {

```

```
    type: String, // URL
    required: true
  },
  isPopular: {
    type: Boolean,
    default: false
  },
  isActive: {
    type: Boolean,
    default: true
  },
  timestamps: true
}
````
```

### ### 3. **\*\*Place Model\*\*** (`Place.js`)

```
````javascript
{
  name: {
    type: String,
    required: true,
    trim: true
  },
  slug: {
    type: String,
    required: true,
    lowercase: true
  },
  cityId: {
    type: ObjectId,
    ref: "City",

```

```
    required: true
  },
  category: {
    type: String,
    enum: ["temple", "fort", "palace", "museum", "nature", "heritage", "religious", "other"],
    required: true
  },
  description: {
    type: String,
    required: true
  },
  history: {
    type: String
  },
  images: {
    type: [String], // Array of URLs
    default: []
  },
  bestTimeToVisit: {
    type: String
  },
  entryFee: {
    type: String // "Free", "₹50", "₹50–₹100"
  },
  location: {
    type: String // Address/landmark
  },
  isActive: {
    type: Boolean,
    default: true
  },
}
```

```
    timestamps: true
  }
  ...

```

4. ****Food Model**** (`Food.js`)

```
````javascript
{
 name: {
 type: String,
 required: true,
 trim: true
 },
 slug: {
 type: String,
 required: true,
 lowercase: true
 },
 cityId: {
 type: ObjectId,
 ref: "City",
 required: true
 },
 type: {
 type: String,
 enum: ["veg", "non-veg", "vegan"],
 required: true
 },
 description: {
 type: String,
 required: true
 },
},

```

```

famousFor: {
 type: String // Why this dish is special
},
approxPrice: {
 type: String // "₹40—₹80", "₹150", "Free"
},
image: {
 type: String, // URL
 required: true
},
isActive: {
 type: Boolean,
 default: true
},
timestamps: true
}
...

```

### ### 5. **\*\*Admin Model\*\*** (`Admin.js`)

```

````javascript
{
  username: {
    type: String,
    required: true,
    unique: true,
    trim: true
  },
  password: {
    type: String,
    required: true
  },
}

```

```

isActive: {
  type: Boolean,
  default: true
},
timestamps: true
}
...

```

🚦 API Endpoints

🌐 Public Routes (No Authentication Required)

States API

| Method | Endpoint | Description | Response |
|--------|-------------------------------|-----------------------|---------------------|
| GET | /api/states | Get all active states | Array of states |
| GET | /api/states/:slug | Get state by slug | Single state object |
| GET | /api/states/:stateSlug/cities | Get cities by state | Array of cities |

Example Response:

```

````javascript
{
 "success": true,
 "message": "States fetched successfully",
 "data": [
 {
 "_id": "64f8a1b2c3d4e5f6a7b8c9d0",
 "name": "Uttar Pradesh",
 "slug": "uttar-pradesh",

```

```
 "description": "The most populous state of India...",
 "image": "https://example.com/up-image.jpg"
 }
]
}
...

```

#### #### \*\*Cities API\*\*

| Method | Endpoint            | Description           | Response           |
|--------|---------------------|-----------------------|--------------------|
| -----  | -----               | -----                 | -----              |
| `GET`  | `/api/cities`       | Get all active cities | Array of cities    |
| `GET`  | `/api/cities/:slug` | Get city by slug      | Single city object |

#### #### \*\*Places API\*\*

| Method | Endpoint                     | Description           | Response            |
|--------|------------------------------|-----------------------|---------------------|
| -----  | -----                        | -----                 | -----               |
| `GET`  | `/api/places`                | Get all active places | Array of places     |
| `GET`  | `/api/places/:slug`          | Get place by slug     | Single place object |
| `GET`  | `/api/places/city/:citySlug` | Get places by city    | Array of places     |

#### #### \*\*Foods API\*\*

| Method | Endpoint                    | Description          | Response           |
|--------|-----------------------------|----------------------|--------------------|
| -----  | -----                       | -----                | -----              |
| `GET`  | `/api/foods`                | Get all active foods | Array of foods     |
| `GET`  | `/api/foods/:slug`          | Get food by slug     | Single food object |
| `GET`  | `/api/foods/city/:citySlug` | Get foods by city    | Array of foods     |

#### #### \*\*Transport API\*\*

| Method | Endpoint         | Description               | Response            |
|--------|------------------|---------------------------|---------------------|
| -----  | -----            | -----                     | -----               |
| `GET`  | `/api/transport` | Get all transport options | Array of transports |

| `GET` | `/api/transport/city/:citySlug` | Get transport by city | Array of transports |

#### #### \*\*Culture API\*\*

| Method | Endpoint | Description | Response |

|-----|-----|-----|-----|

| `GET` | `/api/cultures` | Get all culture data | Array of cultures |

| `GET` | `/api/cultures/:stateSlug` | Get culture by state | Single culture object |

#### #### \*\*Other Services\*\*

| Method | Endpoint | Description | Request Body |

|-----|-----|-----|-----|

| `POST` | `/api/translate` | Language translation | `{ text, from, to }` |

| `GET` | `/api/recommendations` | Travel recommendations | Query params |

| `POST` | `/api/itineraries` | Generate itineraries | `{ preferences, duration }` |

---

#### ### \*\*🔒 Admin Protected Routes (Authentication Required)\*\*

##### #### \*\*Authentication\*\*

| Method | Endpoint | Description | Request Body |

|-----|-----|-----|-----|

| `POST` | `/api/admin/login` | Admin login | `{ username, password }` |

##### \*\*Login Response:\*\*

```
```javascript
```

```
{
```

```
  "success": true,
```

```
  "message": "Admin login successful",
```

```
  "data": {
```

```
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

```
}  
}  
...
```

States Management

| Method | Endpoint | Description | Auth Required |
|----------|---------------------------------------|-----------------------------|---------------|
| ----- | ----- | ----- | ----- |
| `GET` | `/api/admin/states` | Get all states (admin view) | ✔ |
| `GET` | `/api/admin/states/:id` | Get state by ID | ✔ |
| `POST` | `/api/admin/states` | Create new state | ✔ |
| `PUT` | `/api/admin/states/:id` | Update state | ✔ |
| `DELETE` | `/api/admin/states/:id` | Delete state | ✔ |
| `PATCH` | `/api/admin/states/:id/toggle-active` | Toggle state status | ✔ |
| `POST` | `/api/admin/states/bulk-upload` | Bulk upload states | ✔ |

Cities Management

| Method | Endpoint | Description | Auth Required |
|----------|---------------------------------------|-----------------------------|---------------|
| ----- | ----- | ----- | ----- |
| `GET` | `/api/admin/cities` | Get all cities (admin view) | ✔ |
| `GET` | `/api/admin/cities/:id` | Get city by ID | ✔ |
| `POST` | `/api/admin/cities` | Create new city | ✔ |
| `PUT` | `/api/admin/cities/:id` | Update city | ✔ |
| `DELETE` | `/api/admin/cities/:id` | Delete city | ✔ |
| `PATCH` | `/api/admin/cities/:id/toggle-active` | Toggle city status | ✔ |
| `POST` | `/api/admin/cities/bulk-upload` | Bulk upload cities | ✔ |

Places Management

| Method | Endpoint | Description | Auth Required |
|--------|---------------------|-----------------------------|---------------|
| ----- | ----- | ----- | ----- |
| `GET` | `/api/admin/places` | Get all places (admin view) | ✔ |

| | | | | |
|----------|---------------------------------|--------------------|--|--|
| `GET` | `/api/admin/places/:id` | Get place by ID |  | |
| `POST` | `/api/admin/places` | Create new place |  | |
| `PUT` | `/api/admin/places/:id` | Update place |  | |
| `DELETE` | `/api/admin/places/:id` | Delete place |  | |
| `POST` | `/api/admin/places/bulk-upload` | Bulk upload places |  | |

****Foods Management****

| | | | | |
|----------|--------------------------------|----------------------------|--|--|
| Method | Endpoint | Description | Auth Required | |
| ----- | ----- | ----- | ----- | |
| `GET` | `/api/admin/foods` | Get all foods (admin view) |  | |
| `GET` | `/api/admin/foods/:id` | Get food by ID |  | |
| `POST` | `/api/admin/foods` | Create new food |  | |
| `PUT` | `/api/admin/foods/:id` | Update food |  | |
| `DELETE` | `/api/admin/foods/:id` | Delete food |  | |
| `POST` | `/api/admin/foods/bulk-upload` | Bulk upload foods |  | |

****Transport Management****

| | | | | |
|----------|------------------------------------|---------------------------------|---|--|
| Method | Endpoint | Description | Auth Required | |
| ----- | ----- | ----- | ----- | |
| `GET` | `/api/admin/transport` | Get all transports (admin view) |  | |
| `GET` | `/api/admin/transport/:id` | Get transport by ID |  | |
| `POST` | `/api/admin/transport` | Create new transport |  | |
| `PUT` | `/api/admin/transport/:id` | Update transport |  | |
| `DELETE` | `/api/admin/transport/:id` | Delete transport |  | |
| `POST` | `/api/admin/transport/bulk-upload` | Bulk upload transports |  | |

Authentication System

****JWT Authentication Flow****

1. ****Login****: Admin provides username/password
2. ****Token Generation****: Server generates JWT with 1-day expiry
3. ****Token Usage****: Client sends token in Authorization header
4. ****Token Validation****: Middleware validates token for protected routes

****Authentication Middleware**** (`auth.middleware.js`)

```
````javascript
const verifyAdmin = (req, res, next) => {
 try {
 const authHeader = req.headers.authorization;

 if (!authHeader || !authHeader.startsWith("Bearer ")) {
 return res.status(401).json({
 success: false,
 message: "Authorization token missing"
 });
 }

 const token = authHeader.split(" ")[1];
 const decoded = jwt.verify(token, process.env.JWT_SECRET);

 req.adminId = decoded.adminId;
 next();
 } catch (error) {
 return res.status(401).json({
 success: false,
 message: "Invalid or expired token"
 });
 }
};
```

...

### ### \*\*Admin Account Management\*\*

- **\*\*Default Credentials\*\***: `admin` / `admin123`
- **\*\*Creation Command\*\***: `npm run create-admin`
- **\*\*Security Note\*\***: ⚠️ Passwords stored in plain text (needs hashing)

---

## ## 🚀 Server Configuration

### ### \*\*Environment Variables\*\* (`.env`)

```env

Server Configuration

PORT=5000

Database Configuration

MONGO_URI=mongodb+srv://bharatYatra:89792505@bharatyatra.9v7edjt.mongodb.net/bharatyatra?retryWrites=true&w=majority

Authentication

JWT_SECRET=supersecretkey

```

### ### \*\*CORS Configuration\*\*

```javascript

app.use(cors({

 origin: "http://localhost:5173", // Frontend URL

 credentials: true

}));

```

### ### \*\*Request Limits\*\*

```
```javascript
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true, limit: '10mb' }));
```
```

### ### \*\*Database Connection\*\* (`config/db.js`)

```
```javascript
const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB connected");
  } catch (error) {
    console.error("MongoDB connection failed:", error.message);
    process.exit(1);
  }
};
```
```

---

## ## 📦 API Response Format

### ### \*\*Standardized Response Structure\*\* (`utils/apiResponse.js`)

#### #### \*\*Success Response\*\*

```
```javascript
{
  "success": true,
  "message": "Operation completed successfully",

```

```
"data": {  
  // Response data object or array  
}  
}  
...
```

Error Response

```
````javascript  
{
 "success": false,
 "message": "Error description"
}
...
```

#### ### \*\*HTTP Status Codes\*\*

| Code  | Meaning               | Usage                          |
|-------|-----------------------|--------------------------------|
| ----- | -----                 | -----                          |
| `200` | OK                    | Successful GET, PUT, PATCH     |
| `201` | Created               | Successful POST                |
| `400` | Bad Request           | Invalid request data           |
| `401` | Unauthorized          | Missing/invalid authentication |
| `404` | Not Found             | Resource not found             |
| `500` | Internal Server Error | Server-side errors             |

---

## ## 🛠️ Development Setup

#### ### \*\*Prerequisites\*\*

- Node.js (v14 or higher)
- MongoDB Atlas account

- Git

### ### \*\*Installation Steps\*\*

```
```bash
```

1. Clone the repository

```
git clone <repository-url>
```

```
cd BharatYatra/server
```

2. Install dependencies

```
npm install
```

3. Create environment file

```
cp .env.example .env
```

Edit .env with your configuration

4. Create admin account

```
npm run create-admin
```

5. Start development server

```
npm run dev
```

```
```
```

### ### \*\*Available Scripts\*\*

```
```json
```

```
{
```

```
  "start": "node server.js",
```

```
  "dev": "node server.js",
```

```
  "create-admin": "node scripts/createAdmin.js"
```

```
}
```

```
```
```

---

## ## ⚠ Security Considerations

### ### \*\*Current Security Issues\*\*

1. **Plain Text Passwords**: Admin passwords not hashed
2. **Hardcoded Secrets**: JWT secret in environment file
3. **No Rate Limiting**: API vulnerable to abuse
4. **Missing Input Validation**: No request data validation
5. **No HTTPS Enforcement**: HTTP connections allowed

### ### \*\*Security Recommendations

#### #### \*\*Immediate Actions Required

```
```\njavascript\n// 1. Implement password hashing\nconst bcrypt = require('bcrypt');\nconst hashedPassword = await bcrypt.hash(password, 10);\n\n// 2. Add input validation\nconst { body, validationResult } = require('express-validator');\n\n// 3. Implement rate limiting\nconst rateLimit = require('express-rate-limit');\nconst limiter = rateLimit({\n  windowMs: 15 * 60 * 1000, // 15 minutes\n  max: 100 // limit each IP to 100 requests per windowMs\n});\n\n// 4. Add request sanitization\nconst mongoSanitize = require('express-mongo-sanitize');
```

```
app.use(mongoSanitize());
```

```
...
```

```
---
```

🚀 Deployment Guide

**Production Environment Setup**

**Environment Configuration**

```
```env
```

```
Production .env
```

```
NODE_ENV=production
```

```
PORT=5000
```

```
MONGO_URI=<production-mongodb-uri>
```

```
JWT_SECRET=<strong-random-secret>
```

```
```
```

**Server Deployment Options**

****Option A: Heroku Deployment****

```
```bash
```

```
Install Heroku CLI
```

```
npm install -g heroku
```

```
Login and create app
```

```
heroku login
```

```
heroku create bharatyatra-api
```

```
Set environment variables
```

```
heroku config:set MONGO_URI=<your-mongo-uri>
```

```
heroku config:set JWT_SECRET=<your-jwt-secret>
```

```
Deploy
```

```
git push heroku main
```

```
...
```

### **\*\*Option B: Docker Deployment\*\***

```
```dockerfile
```

```
# Dockerfile
```

```
FROM node:16-alpine
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm ci --only=production
```

```
COPY . .
```

```
EXPOSE 5000
```

```
CMD ["npm", "start"]
```

```
...
```

```
---
```

📞 Support & Contact

****Developer:**** Vishal

****Project:**** BharatYatra

****Documentation Version:**** 1.0.0

****Last Updated:**** December 2024

For technical support or questions about this API documentation, please refer to the project repository or contact the development team.

```
---
```

****© 2024 BharatYatra. All rights reserved to Vishal.****